

Arquitectura y Decisiones de Diseño

Enero 2026

1. Arquitectura General

La aplicación sigue una arquitectura basada en el patrón **MVC (Modelo-Vista-Controlador)**, diseñada para separar la lógica de negocio de la interfaz de usuario, facilitando así el mantenimiento y la escalabilidad del sistema.

- **Modelo:** Contiene las clases de dominio (**Gasto**, **Usuario**, **CuentaCompartida**, **Alerta...**). Estas clases encapsulan los datos y las reglas de negocio esenciales.
- **Vista:** Implementada con **JavaFX** y archivos **FXML**. Se han desarrollado dos interfaces: una gráfica (GUI) para el usuario final y una de línea de comandos (CLI) para administración o depuración.
- **Controlador:** El **ControladorAppGastos** actúa como mediador, gestionando las peticiones de las vistas y coordinando las actualizaciones en el modelo y la persistencia.

2. Decisiones de Diseño de Interés

2.1. Gestión de Instancias y Desacoplamiento

Una decisión clave ha sido centralizar la creación de componentes en la clase **Configuracion**. Al implementar esta clase como un *Singleton*, se garantiza que:

- Exista un único **ControladorAppGastos** en toda la ejecución.
- El controlador sea el único con acceso a los **Repositorios**, eliminando la necesidad de que estos sean Singletons por sí mismos. Esto simplifica la jerarquía de objetos y asegura que la persistencia esté controlada desde un solo punto.

2.2. Persistencia Flexible

Se ha optado por una persistencia basada en archivos **JSON** utilizando la librería Jackson. Para evitar el acoplamiento a este formato específico, se ha utilizado el patrón **Adapter**. Esto permite que, si en el futuro se desea migrar a una base de datos SQL, solo sea necesario cambiar la implementación del repositorio sin alterar una sola línea de la lógica de negocio.

2.3. Modelo de Delegación de Eventos

Para mantener la interfaz gráfica sincronizada sin crear dependencias circulares, se ha implementado un sistema de **Delegación de Eventos (Observer)**. El controlador notifica eventos significativos (como **GASTO_AÑADIDO**) y las vistas interesadas se actualizan de forma autónoma. Esto permite, por ejemplo, que al añadir un gasto en una ventana, el gráfico de estadísticas de otra ventana se actualice automáticamente.

2.4. Extensibilidad mediante Estrategias

Para la gestión de repartos en cuentas compartidas y la lógica de las alertas, se ha aplicado el patrón **Strategy**. Esta decisión permite añadir nuevos tipos de reparto (ej. por peso, por consumo) o nuevos periodos de alerta (ej. anual, diario) simplemente creando una nueva clase que cumpla con la interfaz, cumpliendo así con el principio de *Abierto/Cerrado*.

2.5. Integridad de Datos en la Importación

Una decisión de diseño orientada a la consistencia de la información es la validación de duplicados durante el proceso de importación de gastos personales. El sistema garantiza que no se incorporen registros redundantes al repositorio de datos.

- **Validación en Lógica de Negocio:** La comprobación no reside en el importador, sino en el **ControladorAppGastos**, que actúa como filtro verificando si el gasto ya existe antes de procesarlo.
- **Criterio de Identidad:** Se utiliza el identificador único para determinar su existencia previa en el sistema.
- **Consistencia del Repositorio:** Esta medida asegura que las estadísticas de consumo y el histórico de gastos personales reflejen la realidad financiera del usuario sin distorsiones por entradas duplicadas provenientes de archivos externos.

3. Tecnologías Utilizadas

- **Java 21** como lenguaje de programación.
- **JavaFX** para la interfaz de usuario enriquecida.
- **Jackson** para el mapeo de objetos a JSON.
- **Maven** para la gestión de dependencias y construcción del proyecto.